

Practice with the Low Power and Long Range Network LoRa/LoRaWAN

Necessary hardware for the practice

- a LoRa module sending a message on the 868.1 MHz frequency (LoRaSend)
- one LoRa ARM M0 module and one micro-USB cable per student
- a USB multimeter
- a 1200 mAh and 2400 mAh Lithium-ion battery
- 3 solar panels and regulators : 3,4W ; 1.5 W ; 1 W

Presentation of the LoRa/LoRaWAN long-distance network

It is a network designed for connected objects that are isolated or far from the network infrastructure by several kilometers (1 to 40 km). The transmitted signal is easy to understand when received in order to be able to cover long distances. On the other hand, its bitrate is limited to a few Kilobits per second. The signal shape can be configured to focus on distance, robustness, operating time, bitrate or safety.

Comparative table of LoRa/LoRaWAN technology compared to Sigfox, LTE (cellular network)

	LoRaWAN	Sigfox	NB-LTE
Modulation	Spectrum Spreading (CSS)	Ultra Narrow Band, GFSK, BPSK	
Frequency	169, 433, 868 MHz	868 MHz	
Channel Width	125,250, 500 kHz	100 Hz	200 kHz
bitrate	290 bps to 50 Kbps	10 to 1000 bps	20 Kbps
Packet Size	Up to 242 bytes max	8 (DL data) to 12 (UL data) bytes	
Transmission time of 1 packet	~ 1 second	~ 2 seconds	
# message per day	unlimited (duty cycle 1%)	140/day	unlimited (duty cycle)
Correction Code	Code Rate 1 à 4		
Immunity to interference	Very High	Low	Low
Transmitting Power	+20dBm (100 mW)	+20dBm (100 mW)	+20dBm (100mW)
Link budget (Bilan de liaison)	154dB SF12 (-134dBm) SF8 (-124dBm)	151dB	150dB
Receiving distance	30km (in view LoS)	20km (in view LoS)	
Energy Efficiency	Very High	Very High	Average/High
Consumption	12uA sleep 13mA RX (11mcu+2rx) 30mA TX (14dBm) 110mA TX (20dBm) 400mA TX (23dBm) 700mA TX (30dBm)	61mA TX (20dBm)	
Battery Life 2000mAh	105 months (>8 years)	90 months (>7 years)	2 years ?
Security	AES128	No	No
Operator in France	Orange Bouygues (4300 antennas, 86 % of France)	SFR (1500 antennas 91 % of people)	15000 antennas 3G/4G

Cycleo startup in Grenoble (France) created in 2009 this communication technology. Semtech company (US), bought Cycleo in 2012. There are currently 3 companies marketing this product: Semtech SX1272/73 and Hoperf RFM95/98 and Nordic

There is 2 existing components sold by Adafruit:

- Atmega 32u4 microcontroller + LoRa module (RFM95 868MHz) : <https://www.adafruit.com/product/3078>
- ARM Cortex-M0 microcontroller + LoRa module: <https://www.adafruit.com/product/3178>

For this practice we use only ARM Cortex-M0.

For building program, we use the development toolkit called Arduino with version 1.8.X where we add compilation components for our target hardware.

First Exercise on your own computer: Compile « blink » program

Download and install Arduino IDE 1.8.X

Then, installation of the plug-in to compile on the target architectures: adafruit ARM-M0

To install the SAMD compiler for the ARM-M0 processor

Run Arduino tool then File → Preferences

https://adafruit.github.io/arduino-board-index/package_adafruit_index.json

(If there is already content, the URLs must be separated by a comma).

Tools → board type → board management

Install « Arduino SAMD boards (32-bit ARM cortex-M0+) »

then, install « Adafruit SAMD boards »

Choose the target hardware Tool → Board → « adafruit feather M0 express »

Test the blink example, to validate the correct installation of the compiler (File → Example)

Sometimes it can be needed to activate the programming mode doing a fast double-click on the RESET button of the Feather board and view the LED blinking in heartBeat mode.

Second practice: Running a « Hello World » program

Create a new program file and add the follow line in the setup() function, which is run at the launch step.

```
Serial.begin(115200);
```

This line will configure the USB serial port at 115200 bps.

Add the following lines in the loop() function, which is executed each time it ends

```
Serial.println("Hello World"); // This will add the message on the Serial Window at the running step  
delay(3000); // This will add a 3-second delay between each message
```

Then compile and upload this program to test the microcontroller : sketch → upload

If it runs properly, add a print counter in your program to view the values in the monitor screen.

Third practice: Receiving a LoRa signal from a component in the room

Install the Radiohead library to use the LoRa/LoRaWAN communication modules by inserting the file .zip (croquis → include a library → zip file)

<http://www.airspayce.com/mikem/arduino/RadioHead/>

<http://www.airspayce.com/mikem/arduino/RadioHead/RadioHead-1.105.zip>

Creating the reception signal program

```
#include <SPI.h> // The LoRa module includes an interface with SPI bus  
#include <RH_RF95.h> // Radiohead Library for LoRa communications
```

```
#define RFM95_CS 8  
#define RFM95_RST 4  
#define RFM95_INT 3
```

```

RH_RF95 rf95(RFM95_CS, RFM95_INT);

uint8_t buf[255];
uint8_t len = sizeof(buf);

void setup() {
  Serial.begin(115200);

  rf95.init();
  rf95.setModemConfig(RH_RF95::Bw125Cr45Sf128);
                                     // Bandwidth=125kHz Code rate 4/5 Spreading factor =2^7
  rf95.setFrequency(868.1); // 868.1, 868.2 or 868.3
}

void loop() {
  if (rf95.waitAvailableTimeout(4000)) { // Wait for a packet for 4 seconds
    if (rf95.recv(buf, &len)) {
      Serial.println((char*)buf);
      Serial.print("RSSI: ");
      Serial.println(rf95.lastRssi(), DEC);
      Serial.print("SNR: ");
      Serial.println(rf95.lastSNR(), DEC);
    }
  }
  else Serial.println("No reception");
}

```

Take a screenshot of your screen with the serial window on it with succesful reception and put it in your document file.

Forth practice: transmit a LoRa signal with a specific spreading factor

In the setup function, add the configuration of the transmit power :

```
rf95.setTxPower(13, false); // +13dBm
```

In order to send periodically a packet each 2 seconds

```

void loop() {
  message="LoRa Practice #";
  message+=i;
  i++;
  message.toCharArray((char*)buf, message.length()+1);
  buf[message.length()+1] = 0;
  rf95.send((uint8_t *)buf, message.length()+2);
  rf95.waitPacketSent();
  Serial.print("Sent Message:");
  Serial.println(message);
  delay(2000);
}

```

Add a LED blink for each transmission, in order to visualise the time to send.

When your program is running, connect the multimeter on the USB cable and read the maximum instantaneous current consumption (mA) and the energy consumed (mAh) for 2 minutes for the following transmission powers: + 20dBm + 14dBm and + 8dBm. Read the corresponding RSSI on the receiver.

Modify your program to fill the following table (give the config function)

State	Transmit power	Current - mA	RSSI on receive module (around 1 m)	Time - second
mcu only		15		
mcu+transit 32 bytes in SF12	+20dBm			
mcu+transmit 32 bytes in SF10	+20dBm			
mcu+transmit 256 bytes in SF8	+20dBm			
mcu+transmit 256 bytes in SF7	+20dBm			
mcu+transit 32 bytes in SF12	+14dBm			
mcu+transmit 32 bytes in SF10	+14dBm			
mcu+transmit 256 bytes in SF8	+14dBm			
mcu+transmit 256 bytes in SF7	+14dBm			
mcu+transit 32 bytes in SF12	+8dBm			
mcu+transmit 32 bytes in SF10	+8dBm			
mcu+transmit 256 bytes in SF8	+8dBm			
mcu+transmit 256 bytes in SF7	+8dBm			
mcu with LoRa receive				
mcu with LoRa in sleep mode				

Fifth practice: Solar Panel only.

Run your microcontroller and send a message at +8dBm with only a solar panel as energy unit and put the LED on before to send to be sure that your message is sent and verify that you receive this message on an other microcontroller. If it works try with a smaller solar panel and/or growth the transmit power.

Sixth practice: Periodic running of deep-sleep mode.

To manage the running and deep-sleep modes, the Real Time Clock (RTC) module must be defined by including the following library and defining a global variable:

```
#include <RTCZero.h>
RTCZero rtc;
```

In the setup function, the RTC must be configured as follows:

```
rtc.begin();
rtc.enableAlarm(rtc.MATCH_SS); // Match seconds only
rtc.attachInterrupt(alarmMatch); // Attach function to interrupt
```

At the beginning of the loop function, add a double blink of 100ms of the LED to observe the behavior of the program, and at the end instead of the delay function, add the following 2 lines:

```
rtc.setAlarmSeconds((rtc.getSeconds()+5)%60);
rtc.standbyMode(); // Sleep until next alarm match
```

After a *standbyMode*, the synchronization of the *Serial* port is lost and messages are no longer sent back to the console. The variables keep their values when the processor is restarted. Using the multimeter, measure instantaneous and cumulative consumption. Repeat the exercise with the following parameters:

```
rf95.setModemConfig(RH_RF95::Bw125Cr48Sf4096); // SF=12, 2^12=4096
```

If you measure the current consumption take care to close the LoRa unit using the *sleep()* method on the *rf95* object.

It can be necessary to do a double click on the reset button on the board in order to stop the deep sleep mode and force heartbeat blinking to force the writing program mode.

Seventh practice: Measure the battery level

Find in the web the function to print the battery level.
What type of operation is needed to measure this value ?

Eighth practice: Send only when the data is different from the previous one

To save energy, reduce the number of messages sent by removing "unnecessary" information. To do this, before activating the LoRa interface and transmitting, it is necessary to check that the data sent is very different from the previous one. To do this, draw a random value in 0 or 1 and send the value only if it is different from the previous one.